

Spring 2026 MSML606 HW3 Extra Credit Project (deadlines: 3/07 for proposal and 3/25 for final submission)

Important! To be eligible for this extra credit, you must submit your project proposal on time. If a proposal is not submitted by the deadline, the rest of the assignment will not be graded, and no bonus points will be awarded.

Objective

The goal of this project is to move beyond the “clean” datasets used in class (integers, simple strings) and apply core algorithms to the complexity and noise of real-world data. You will explore how data preprocessing and algorithmic modifications are necessary to handle missing keys, varied data types (ASCII, floating-point), and metadata.

Overview

1. **Algorithmic Scope:** Implement a functional application using one of the following algorithms:
 - Hashing: Applications for indexing, frequency counting, or feather hashing
 - Trees: General Binary or n -ary trees for hierarchical data
 - Search Trees: Binary Search Trees (BST) or Balanced Trees (AVL, Red-black trees)
2. **Data requirements:** Your application must process real-world data meeting these criteria:
 - Scale: A minimum of 10,000 items
 - Complexity: The data must include multiple metadata fields, not just the primary keys
 - Source types:
 - Web-crawled data
 - In-the-wild datasets (unfiltered)
 - Noisy, curated datasets (containing inconsistent formatting or missing values)
3. **Core challenges to address:**
 - Preprocessing: What transformations are required to make the data functional for your chosen algorithm?
 - Noise handling: How does your algorithm react to “imperfect” data (e.g., special symbols, missing keys, or unexpected data types)?
 - Efficiency: How does your chosen data structure perform when the key count exceeds 10,000?
4. **Examples:**

- Recipe Hierarchies: Visualize recipes in a tree structure ([example](#))
- Subreddit Search: Efficiently search and categorize Reddit group names using a BST
- Feature Hashing: Indexing frequency counts or metadata associations for a large-scale text or image dataset
- Hierarchical navigation: Tree-based visualization of family tree, academic citation

Project Policies and AI Usage:

This assignment adheres to all course policies specified in the syllabus, with the following project-specific modifications:

- Prohibited AI Usage:
 - Do not use AI to “start” your logic or generate your initial approach, or scaffold your code template
- Manual authorship and originality requirement:
 - The following sections must be authored and typed manually by you:
 - Data handling: All code related to preprocessing, data structure implementation, and data access logic
 - Core algorithm: Your chosen algorithmic implementation (e.g., Hashing, trees, BST, etc.).
 - Documentation: All code comments and the project README.
- Permitted AI Usage: You may use AI tools to assist **only** with the following:
 - User Interface (UI) : AI policy is waived for UI components (i.e., copy-pasting or direct generation is allowed). **Condition**: you must manually write the UI specification and submit it with your project.
 - Non-core logic: You may use AI for auxiliary tasks not related to data handling or the core algorithm.
- NOTE: The primary objective of this project is to implement and test algorithms on real-world data. The complexity or “fanciness” or your application is not a priority for this assignment.

Timeline & requirements

Proposal submission (deadline 3/07):

- Submit a max 1-2 page document or slide deck (ELMS)
- Include team composition (individual or groups of up to 3)
- Specify the dataset and brief description of the application

Final Submission (deadline 3/25):

- Share on ELMS (ELMS->discussion->Project1):
 - Post a 3-minute video demonstration of your application
 - Respond to at least 2 other posts
- GitHub repository
 - Source code and setup instructions (how to run the example)
 - Project website (if applicable)
 - Include your original proposal and presentation files in the repo.

An opportunity to present your work may be offered if time permits.

Target audience:

The final project (GitHub and Video) must be designed for a non-CS audience. The goal is for any reader without a CS background to understand exactly what the application is, and how to use it without spending effort beyond watching your video or reading your documentation.

Note on Extra Credit:

Your earned points will be aggregated and weighted with other bonus points (e.g., Hashing Practice problems). This total will be added to your raw midterm grade before any final scaling is applied. Please note that bonus points do not have a 1:1 correspondence with the final 100-point course scale.

Rubric: Maximum Score: 21 (Raw Marks)

Bonus value: 3 points (your score will be scaled linearly)

Problem	Criteria	Marks
Proposal (due 3/7)	- Submitted on time (must have) - Correct format with 1-2 pages/slides (1 marks) - Team composition clearly stated (1 marks) - Dataset identified and described (2 marks) - Application idea is clear and uses the main algorithm meaningfully (2 marks)	6
Code & GitHub Repository (due 3/25)	- Main algorithm is correctly implemented and central to the app (2 marks) - Code is functional (i.e. runs without errors per setup instructions) (must have) - Instructions are clear and code is readable and well-commented (2 marks) - Proposal and presentation files included in repo (1 marks)	6

	- Evaluation based on commit history, code organization, and clarity of documentation (1 marks)	
Video Presentation (due 3/25)	- Demonstrates all core features of the application (2 marks) - Accessible to a non-CS audience with no unexplained jargon; Target audience awareness across all submissions (2 marks) - Within 3-minute time limit (1 marks)	5
Peer Engagement (due 3/25)	- Responded to at least 2 classmates posts (2 marks) - Responses are substantive, not just “great job!” (<u>must have</u>)	2
Originality & Academic Integrity	- <u>State of using AI/external sources (must have)</u> - No AI used for core logic or initial approach (must have) - Code, comments, and README are manually authored (must have)	-
Creativity	Originality in the application or chosen dataset	2

- The items marked as “must-have” are strict prerequisites. If any must-have requirement is missing or unmet, the project will receive a grade of 0.
- The marks listed in the rubric represent the maximum possible score for each category. Final marks will be awarded based on the depth, accuracy, and quality of your work.

NB: Problem difficulty and UI aesthetics are not evaluated!